

Many To Many XML

(Advised to do it by your own)

Many To Many Annotation

QueryNN.java

```
package hibernatemapping;

import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.ManyToMany;

@Entity
public class QueryNN {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(length = 10)
    private int id;
    @Column(length = 20)
    private String qname;

    @ManyToMany(cascade = CascadeType.ALL)
    private List<ResponseNN> responses;

    public int getId() {
        return id;
    }

    public String getQname() {
        return qname;
    }

    public List<ResponseNN> getResponses() {
        return responses;
    }

    public void setId(int id) {
```

```

        this.id = id;
    }

    public void setQname(String qname) {
        this.qname = qname;
    }

    public void setResponses(List<ResponseNN> responses) {
        this.responses = responses;
    }

}

```

ResponseNN.java

```

package hibernatemapping;

import java.util.List;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.ManyToOne;

@Entity
public class ResponseNN {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(length = 10)
    private int id;

    @Column(length = 30)
    private String responsestr;

    @Column(length = 15)
    private String postedby;

    @ManyToMany(mappedBy = "responses")
    private List<QueryNN> queries;

    public int getId() {
        return id;
    }
}

```

```

    public String getResponsestr() {
        return responsestr;
    }

    public String getPostedby() {
        return postedby;
    }

    public void setId(int id) {
        this.id = id;
    }

    public void setResponsestr(String responsestr) {
        this.responsestr = responsestr;
    }

    public void setPostedby(String postedby) {
        this.postedby = postedby;
    }

    public List<QueryNN> getQueries() {
        return queries;
    }

    public void setQueries(List<QueryNN> queries) {
        this.queries = queries;
    }
}

```

hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.current_session_context_class">thread</property>
        <property
name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriv
er</property>

```

```

        <property
name="hibernate.connection.password">oracle</property>
        <property
name="hibernate.connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
roperty>
        <property
name="hibernate.connection.username">system</property>
        <property
name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</prope
rty>
        <property name="hbm2ddl.auto">create</property>
        <property name="show_sql">>true</property>

    </session-factory>
</hibernate-configuration>

```

AppQRNN.java

```

package hibernatemapping;

import java.util.ArrayList;
import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class AppQRNN {
    public static void main(String[] args) {

        QueryNN q1 = new QueryNN();
        q1.setQname("Query1N");

        QueryNN q2 = new QueryNN();
        q2.setQname("Query2N");

        ResponseNN r1 = new ResponseNN();
        r1.setResponsestr("Response1N"); r1.setPostedby("Joe");

        ResponseNN r2 = new ResponseNN();
        r2.setResponsestr("Response2N"); r2.setPostedby("John");

        List<QueryNN> listq = new ArrayList<QueryNN>();
        listq.add(q1); listq.add(q2);
    }
}

```

```

List<ResponseNN> listr = new ArrayList<ResponseNN>();
listr.add(r1); listr.add(r2);

q1.setResponses(listr);
q2.setResponses(listr);

r1.setQueries(listq);
r2.setQueries(listq);

Configuration conf = new
Configuration().configure("hibernate.cfg.xml")
    .addAnnotatedClass(QueryNN.class)
    .addAnnotatedClass(ResponseNN.class);

ServiceRegistry sr = new ServiceRegistryBuilder()
    .applySettings(conf.getProperties())
    .buildServiceRegistry();
SessionFactory sf1 = conf.buildSessionFactory(sr);
Session s1 = sf1.openSession();
s1.beginTransaction();

s1.save(q1);
s1.save(q2);

s1.getTransaction().commit();
}
}

```

```
eclipse-workspace - MavenProjectN/src/main/java/hibernate.cfg.xml - Eclipse IDE
File Edit Source Navigate Search Project Run Window Help

<terminated> AppQRNN [Java Application] C:\Users\HP\pZ\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_18.0.1.v20220515-1614\jre\bin\javaw.exe (20-Jul-2023, 4:58:44 pm - 4:58:54 pm) [pid: 15568]

Hibernate: drop table ResponseNN cascade constraints
Hibernate: drop sequence hibernate_sequence
Hibernate: create table QueryNN (id number(10,0) not null, qname varchar2(20 char), primary key (id))
Hibernate: create table QueryNN_ResponseNN (queries_id number(10,0) not null, responses_id number(10,0) not null, postedby varchar2(15 char), responsestr varchar2(150 char))
Hibernate: create table ResponseNN (id number(10,0) not null, postedby varchar2(15 char), responsestr varchar2(150 char))
Hibernate: alter table QueryNN_ResponseNN add constraint FK69E0CA1821B8354A foreign key (queries_id) references QueryNN (id)
Hibernate: alter table QueryNN_ResponseNN add constraint FK69E0CA18AC47B2FB foreign key (responses_id) references ResponseNN (id)
Hibernate: create sequence hibernate_sequence
Jul 20, 2023 4:58:53 PM org.hibernate.tool.hbm2ddl.SchemaExport execute
INFO: HH000230: Schema export complete
Hibernate: select hibernate_sequence.nextval from dual
Hibernate: select hibernate_sequence.nextval from dual
Hibernate: select hibernate_sequence.nextval from dual
Hibernate: select hibernate_sequence.nextval from dual
Hibernate: insert into QueryNN (qname, id) values (?, ?)
Hibernate: insert into ResponseNN (postedby, responsestr, id) values (?, ?, ?)
Hibernate: insert into ResponseNN (postedby, responsestr, id) values (?, ?, ?)
Hibernate: insert into QueryNN (qname, id) values (?, ?)
Hibernate: insert into QueryNN_ResponseNN (queries_id, responses_id) values (?, ?)
Hibernate: insert into QueryNN_ResponseNN (queries_id, responses_id) values (?, ?)
Hibernate: insert into QueryNN_ResponseNN (queries_id, responses_id) values (?, ?)
Hibernate: insert into QueryNN_ResponseNN (queries_id, responses_id) values (?, ?)
```

```
SQL Plus

SQL> select * from QueryNN;

   ID QNAME
-----
    1 Query1N
    4 Query2N

SQL> select * from ResponseNN;

   ID POSTEDBY      RESPONSESTR
-----
    2 Joe          Response1N
    3 John         Response2N

SQL> select * from QueryNN_ResponseNN;

QUERIES_ID RESPONSES_ID
-----
    1         2
    1         3
    4         2
    4         3

SQL> select * from QueryNN;

   ID QNAME
-----
    4 Query2N
```

AppQRNNFetch.java

package hibernatemapping;

```

import java.util.List;

import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class AppQRNNFetch {
    public static void main(String[] args) {
        Configuration conf = new
Configuration().configure("hibernate.cfg.xml")
                .addAnnotatedClass(QueryNN.class)
                .addAnnotatedClass(ResponseNN.class);

        ServiceRegistry sr = new ServiceRegistryBuilder()
                .applySettings(conf.getProperties())
                .buildServiceRegistry();
        SessionFactory sf1 = conf.buildSessionFactory(sr);
        Session s1 = sf1.openSession();
        s1.beginTransaction();

        Query query = s1.createQuery("from QueryNN");

        List<QueryNN> list = query.list();

        for(QueryNN q : list) {
            System.out.println("Query::");
            System.out.println(q.getId());
            System.out.println(q.getQname());

            List<ResponseNN> listres = q.getResponses();

            for(ResponseNN res : listres) {
                System.out.println("Response::");
                System.out.println(res.getId());
                System.out.println(res.getResponsestr());
                System.out.println(res.getPostedby());
            }
            System.out.println("=====");
        }
        System.out.println("*****");

        query = s1.createQuery("from ResponseNN");
    }
}

```

```

List<ResponseNN> listr = query.list();

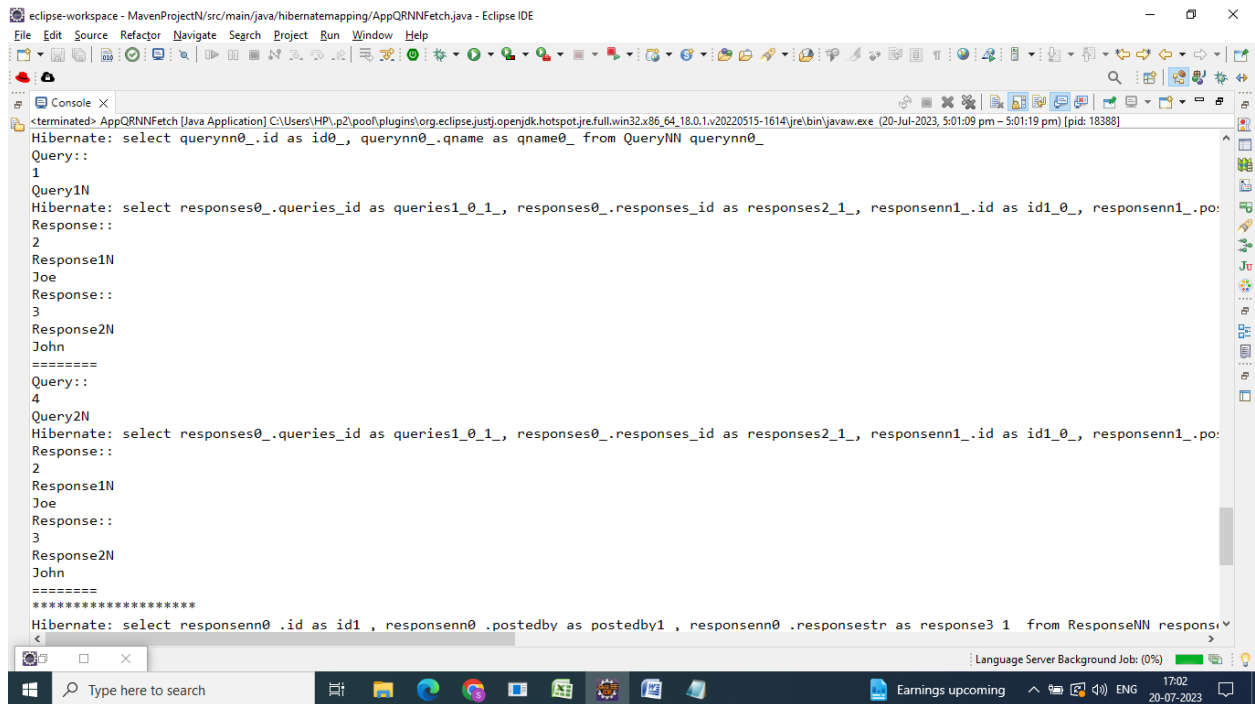
for(ResponseNN res : listr) {
    System.out.println("Response::");
    System.out.println(res.getId());
    System.out.println(res.getResponsestr());
    System.out.println(res.getPostedby());

    List<QueryNN> listq = res.getQueries();

    for(QueryNN q : listq) {
        System.out.println("Query::");
        System.out.println(q.getId());
        System.out.println(q.getQname());
    }
    System.out.println("=====");
}

s1.getTransaction().commit();
}
}

```



The screenshot shows the Eclipse IDE with a Java application running. The console output is as follows:

```

eclipse-workspace - MavenProjectN/src/main/java/hibernatemapping/AppQRNNFetch.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
<terminated> AppQRNNFetch [Java Application] C:\Users\HP\p2\poo\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.18.0.1.v20220515-1614\jre\bin\javaw.exe (20-Jul-2023, 5:01:09 pm - 5:01:19 pm) [pid: 18388]
Hibernate: select querynn0_.id as id0_, querynn0_.qname as qname0_ from QueryNN querynn0_
Query::
1
Query1N
Hibernate: select responses0_.queries_id as queries1_0_1, responses0_.responses_id as responses2_1_, responsenn1_.id as id1_0_, responsenn1_.po:
Response::
2
Response1N
Joe
Response::
3
Response2N
John
=====
Query::
4
Query2N
Hibernate: select responses0_.queries_id as queries1_0_1, responses0_.responses_id as responses2_1_, responsenn1_.id as id1_0_, responsenn1_.po:
Response::
2
Response1N
Joe
Response::
3
Response2N
John
=====
Hibernate: select responsenn0_.id as id1_, responsenn0_.postedby as postedby1_, responsenn0_.responsestr as response3_1 from ResponseNN responsi

```


QueryNN.java

@ManyToMany(cascade = CascadeType.**DETACH**)

AppQRNNDelete.java

```
package hibernatemapping;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class AppQRNNDelete {
    public static void main(String[] args) {
        Configuration conf = new
Configuration().configure("hibernate.cfg.xml")
                .addAnnotatedClass(QueryNN.class)
                .addAnnotatedClass(ResponseNN.class);

        ServiceRegistry sr = new ServiceRegistryBuilder()
                .applySettings(conf.getProperties())
```

```

        .buildServiceRegistry();
SessionFactory sf1 = conf.buildSessionFactory(sr);
Session s1 = sf1.openSession();
s1.beginTransaction();

Object query = s1.get(QueryNN.class, 1);

if(query != null) {
    s1.delete(query);
    System.out.println("Query deleted");
}

s1.getTransaction().commit();
}
}

```

SQL Plus

SQL> select * from QueryNN;

ID	QNAME
4	Query2N

SQL> select * from ResponseNN;

ID	POSTEDBY	RESPONSESTR
2	Joe	Response1N
3	John	Response2N

SQL> select * from QueryNN_ResponseNN;

QUERIES_ID	RESPONSES_ID
4	2
4	3

SQL>